Constraint Satisfaction Problems

Professor Rick Valenzano

Toronto Metropolitan University

CPS721: Artificial Intelligence I

Learning Objectives

Students should be able to

1. Identify the structure of a Constraint Satisfaction Problem
2. Implement generate and test on a simple problem
3. Identify ways to interleave value generation and testing to improve performance

Toronto Metropolitan University

Prolog Output

- Two important predicates for output

`write("Some String")` or `write(Term)`

- Writes the string or term to output

`nl`

- Writes a newline to output

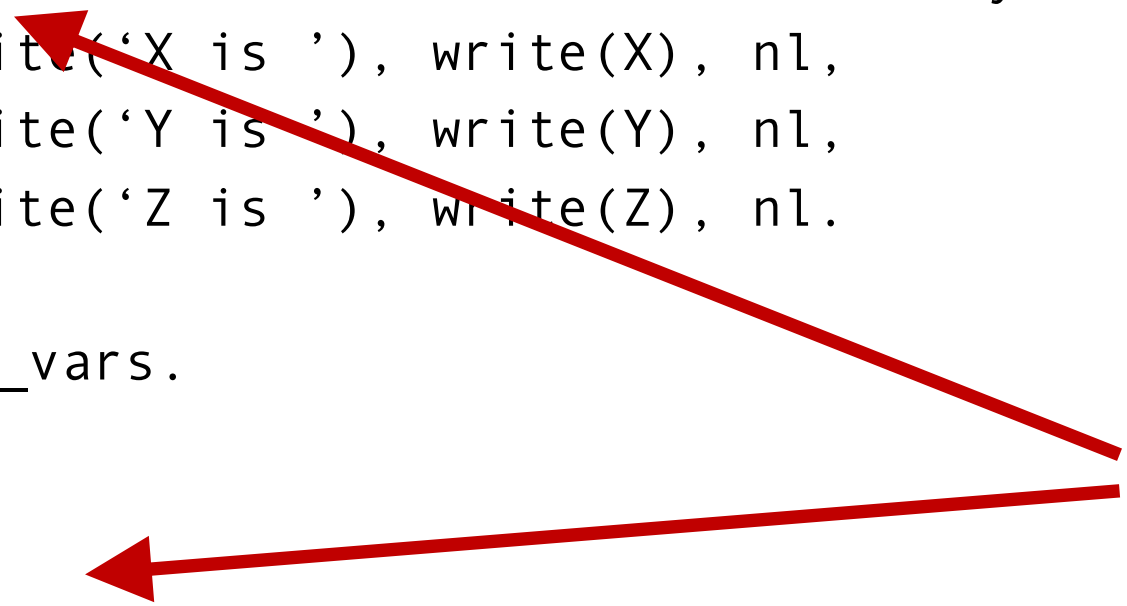
Toronto Metropolitan University

Example Output

```
print_vars :- X = 5, Y = wow, Z = hurray,
      write('X is '), write(X), nl,
      write('Y is '), write(Y), nl,
      write('Z is '), write(Z), nl.

?- print_vars.

X is 5
Y is wow
Z is hurray
```



No variables in the head, so the usual X= …, Y = …, Z = … lines are suppressed

Toronto Metropolitan University

Hospital Rostering

- Have a set of employees, with different skills (Doctors, nurses, etc.)
- Have staffing requirements for different times of the day
- Have a list of constraints that must be handled
  - No one can work more than 16 hours in a row
  - James wants Friday off for vacation

- Find a schedule for the next two weeks that satisfies these

Toronto Metropolitan University

Baseball Scheduling

- There are 30 MLB teams, each plays 162 games
- Need a schedule such that satisfies certain constraints
  - Can’t use the Rogers Centre on certain dates
  - Teams can only be away from home x days in a row

- Find a schedule to satisfy these constraints

Toronto Metropolitan University

Constraint Satisfaction Problems

- Many problems can be represented as follows
  - A set of choices to be made *(ie.* who is on shift tomorrow*)*
  - Those choices must satisfy some set of constraints

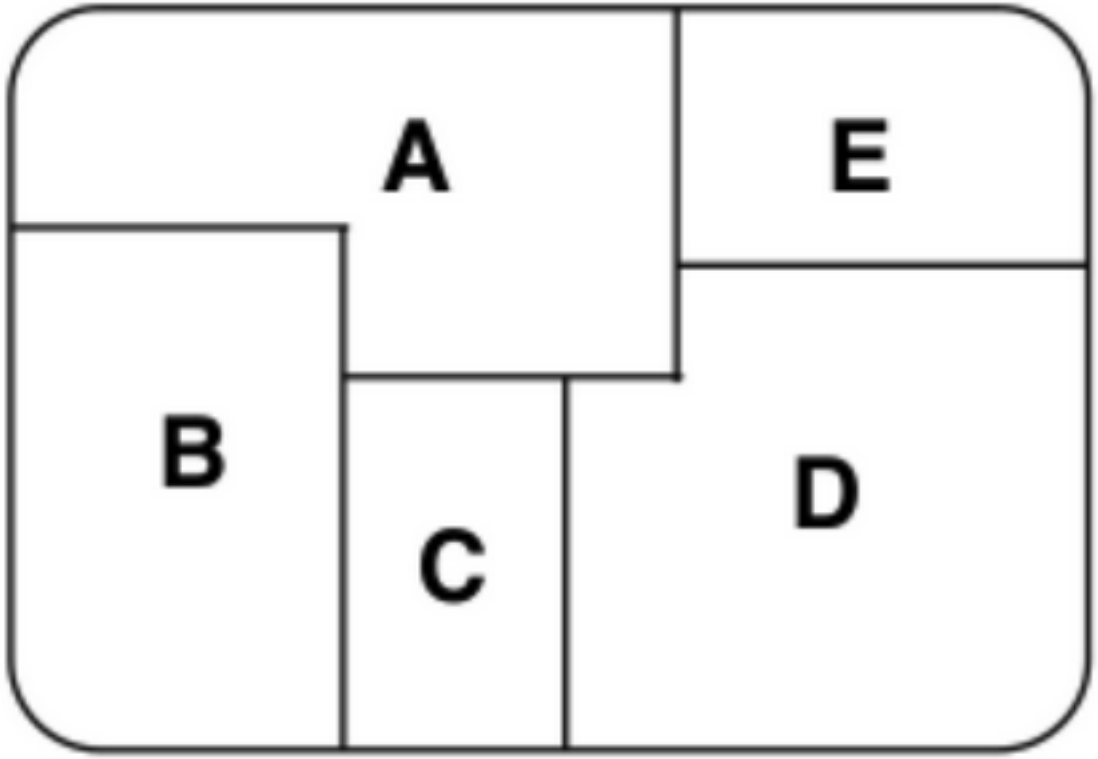
- These are hard reasoning tasks
  - Can be very large and complicated
  - Humans are (sort-of) good at these if the problem isn't too big

- Can also have "quality" metric (such as cost or distance travelled)
  - Beyond the scope of this course

Toronto Metropolitan University

Map Colouring

- Assign colours so that adjacent countries aren't the same colour
  - Three colours: red, blue, white
- How many possible assignments?

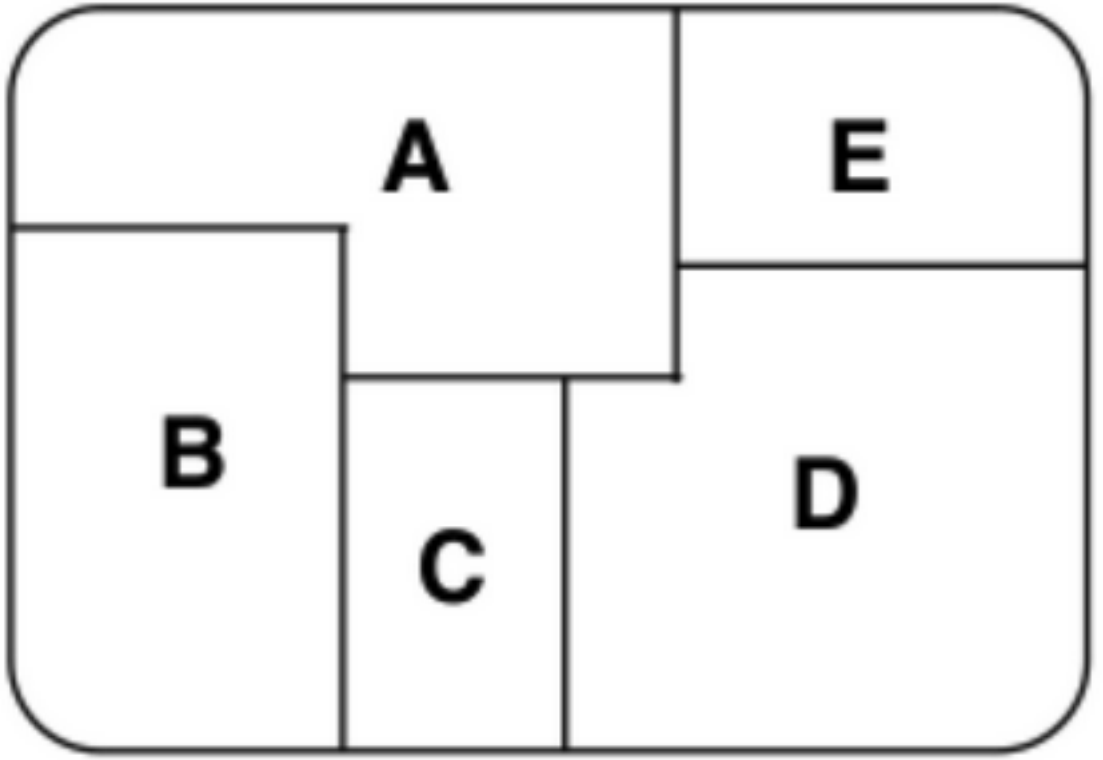


Toronto Metropolitan University

Map Colouring

```
colour(red).
colour(blue).
colour(white).

solve([A, B, C, D, E]) :-
```



Toronto Metropolitan University

Generate and Test

- This is an example of **generate and test**

1. Generate a set of choices
   (ie. `A = red, B = white, C= blue, D = red, E = white`)

2. Test if the constraints are satisfied
   (ie. are adjacent countries assigned the same colour)

3. Backtrack to step 1 if the constraints are not satisfied

Toronto Metropolitan University

Scaling up Map Colouring

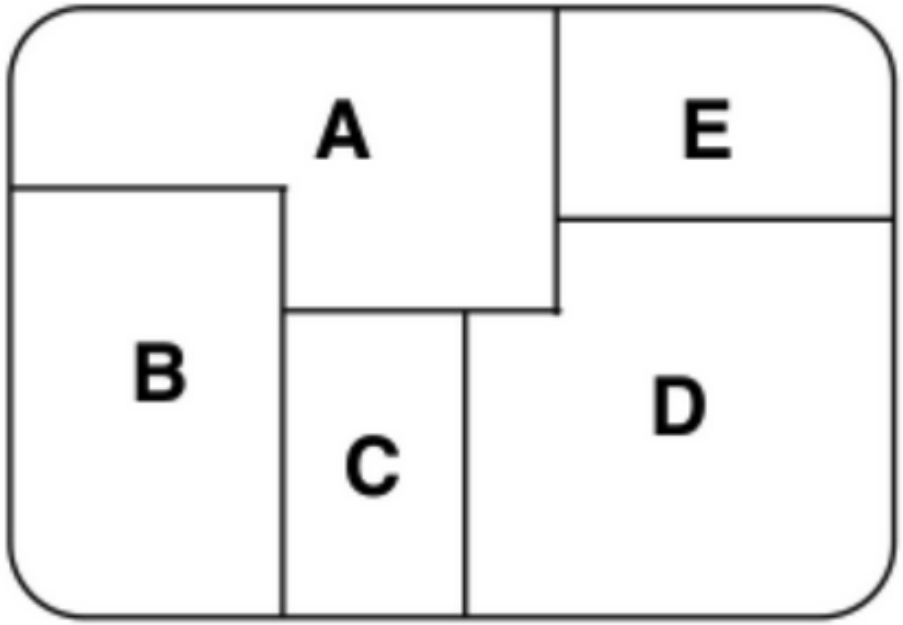
- How many colours do we need?
- Would this scale up to a map of the world (~200)?
  - ~4^{200} possible assignments for 4 colours
  - As comparison, there are ~10^{80} atoms in the universe
  - If we could generate and test 10^{80} per second, would still take trillions of years
- Need a smarter approach

Toronto Metropolitan University

Using a Dependency Graph

- The main idea is to start checking constraints as soon as possible
  - Trying to **prune** possibilities

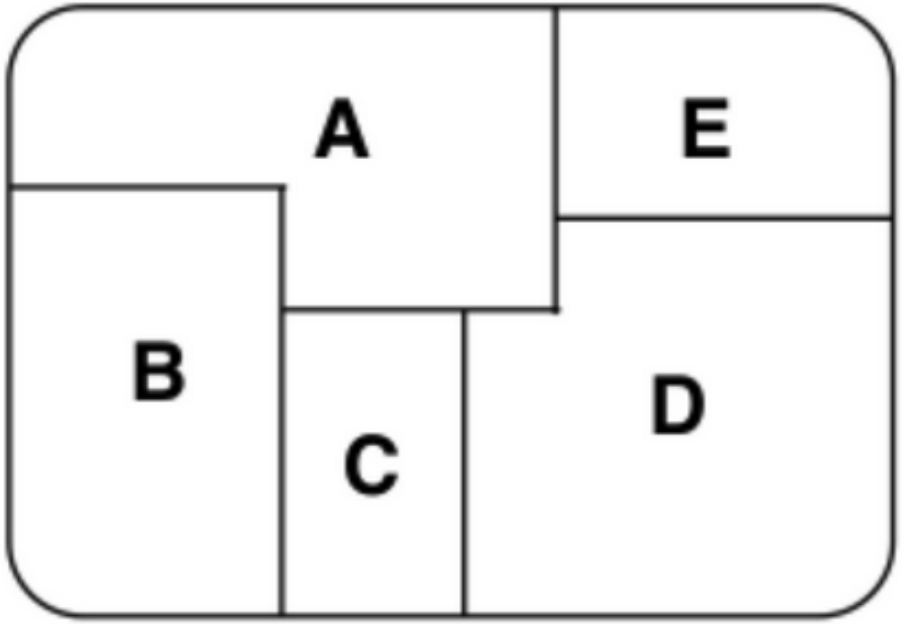
- If I set A to red, what does that say about B, C, D, E?

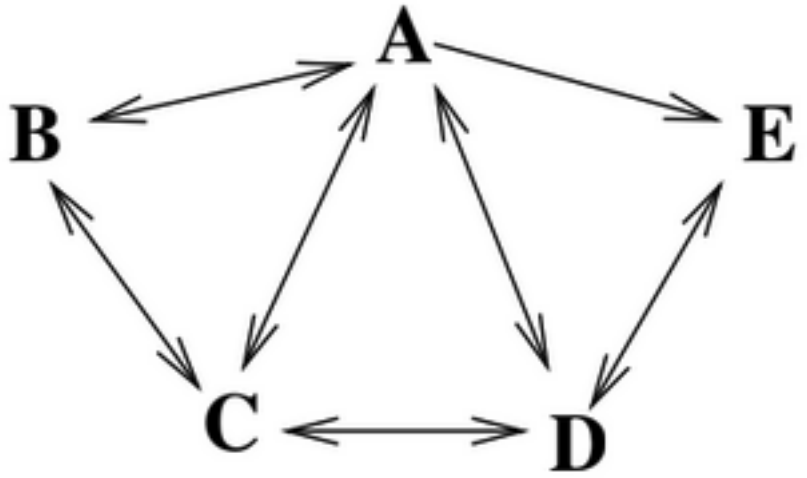


Toronto Metropolitan University

Using a Dependency Graph

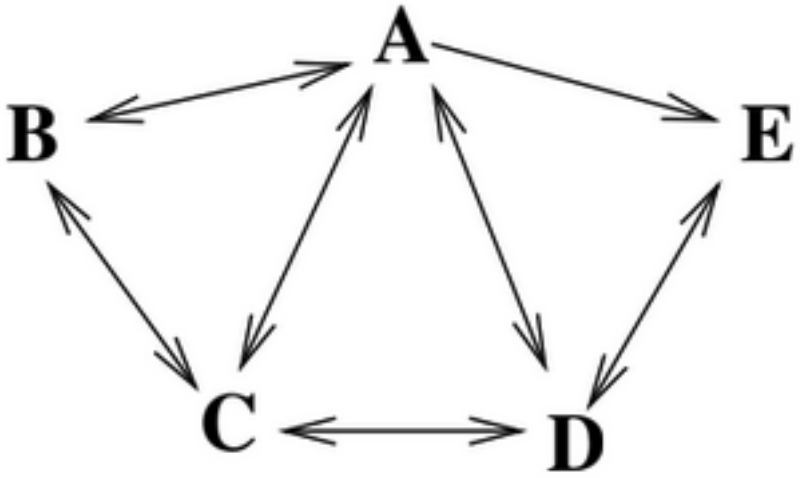
- Can think of the constraints as a graph





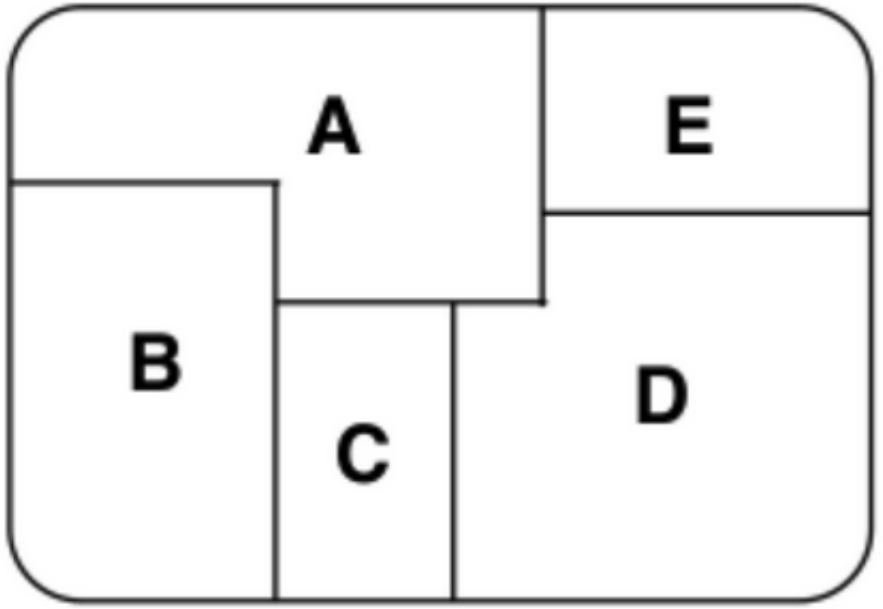
Toronto Metropolitan University

Interleaving Generate and Test



- Partially generate, then test, then generate some more, etc.

Map Colouring




```
colour(red).  colour(blue). colour(white).

solve([A, B, C, D, E]) :-
      colour(A), colour(B), not A = B,
      colour(C), not C = B, not C = A,
      colour(D), not C = D, not C = A,
      colour(E), not E = D, not E = A.

find_and_print :- solve([A, B, C, D, E),
      write("The colour of A is "), write(A), nl,
      …
      write("The colour of E is "), write(E), nl.
```

Toronto Metropolitan University

General CSP Approach (Generate and Test)

solve(ListOfVariables) :-

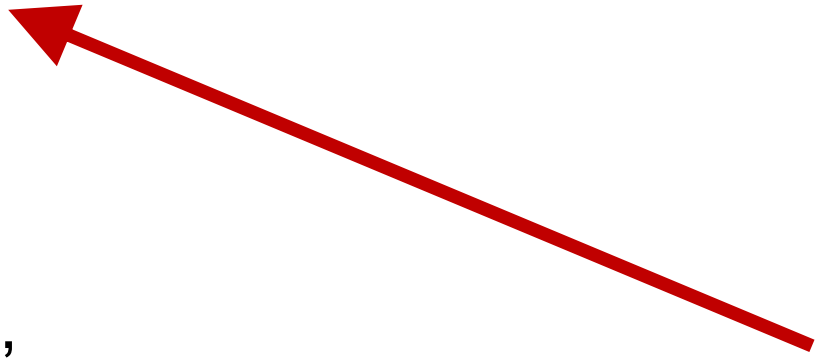
domain$_1$(Var$_1$), domain$_2$(Var$_2$), …, domain$_n$(Var$_n$),

constraint$_1$(Var1, …, Var$_n$), …, constraint$_m$(Var1, …, Var$_n$).

Optionally

solve_and_print :- solve(ListOfVariables),

write("The value of Var1 is "), write(Var$_1$), nl, …



Usually can interleave for better performance

Toronto Metropolitan University

Learning Objectives

Students should be able to

1. Identify the structure of a Constraint Satisfaction Problem
2. Implement generate and test on a simple problem
3. Identify ways to interleave value generation and testing to improve performance

Toronto Metropolitan University